Storage Classes

Introduction....2
Types of Storage Classes in C....3
1) Automatic Storage Class (auto)....3
2)Register Storage Class (register)....5
3)Static Storage Class (static)....6
4)Extern Storage Class (extern)....8
Practice questions....11

Introduction

Storage classes define the **behavior and properties of variables** in a program.

Key Concepts

1. Life (Lifetime)

- Refers to **how long a variable exists in memory**
- Determines the **creation and destruction** of a variable

2. Scope

- Refers to **where a variable can be accessed** in the program
- Defines the **visibility of a variable**

Important Point

- A variable must **exist in memory** to be accessed
- Even if it exists, it may **not be accessible everywhere** due to scope
- Accessing a variable outside its scope causes a **compile-time error**

Types of Storage Classes in C

There are **four storage classes**:

1. auto
2. register
3. static
4. extern

1) Automatic Storage Class (auto)

Key Points

- Default storage class for **local variables**
- Stored in **stack memory**
- Lifetime: **Only within block/function**
- Scope: **Block level**
- Default value: **Garbage**

Important Concepts

- Variables are **created when function/block starts**
- Automatically **destroyed when block ends**
- auto keyword is **optional**

Block Scope Behavior

- Inner block variable **overrides outer variable**
- If not declared in current block → access **outer block variable**

Example Understanding

```c
auto int a = 10;

{

   auto int a = 20;

   printf("%d", a);    // 20 (inner block)

}

printf("%d", a);        // 10 (outer block)
```

Example

```c
#include <stdio.h>

void fun()
{
  auto int x = 5;
```

```
    printf("%d\n", x);
}

int main()
{
    fun();
    fun();
    return 0;
}
```

Output

5

5

Explanation

- auto int x = 5; → declares a **local (auto) variable**
- auto is **default** for all local variables (keyword optional)
- x is stored in **stack memory**

Execution Steps

- main() calls fun()
- **New stack frame created**
- x is created and initialized to 5
- Value 5 is printed
- Function ends → **x is destroyed**
- fun() called again:
    - New x is created again
    - Prints 5

Important Notes

- Local variables **cannot be accessed outside function**
- Stored in **stack → part of stack frame**
- Must initialize variables → otherwise **garbage value**

2)Register Storage Class (register)

Key Points

- register variables are requested to be stored inside fast CPU registers instead of RAM for quicker access and execution.
- Since CPU registers are limited, the compiler stores the variable in a register only if a free register is available.

Important Concepts

- Declared using register keyword
- Lifetime: **Block**
- Scope: **Block**
- Default value: **Garbage**

Example

```c
#include <stdio.h>

int main()
{
  register int i;

  for(i = 1; i <= 5; i++)
  {
     printf("%d ", i);
  }

  return 0;
}
```

Output

1 2 3 4 5

Note : You cannot use the address operator (&) with a register variable.

3)Static Storage Class (static)

Definition

- static can be used with variables and functions
- It controls lifetime and scope
- Similar to global variables but with restricted visibility

Static Variables

Types

1. Local Static Variable
2. Global Static Variable

1. Local Static Variable

Key Points

- Declared inside a function
- Lifetime: Entire program
- Scope: Within that function only
- Default value: 0
- Initialized only once
- Value is retained between function calls

Example

```c
#include <stdio.h>

void fun()
{
  static int a = 1;
  printf("%d ", a);
  a++;
}

int main()
{
  fun();
  fun();
  fun();
  return 0;
}
```

Output

1 2 3

2. Global Static Variable

Key Points

- Declared outside all functions
- Lifetime: Entire program
- Scope: Only within that file
- Cannot be accessed from other files

Static Function

Definition

- A function declared with static
- Its scope is limited to the file in which it is declared

Key Points

- Cannot be called from other files
- Used for data hiding / encapsulation
- Helps avoid name conflicts in large programs

Example

```c
#include <stdio.h>
static void display()
{
   printf("Hello Static Function");
}
int main()
{
   display();   // valid (same file)
   return 0;
}
```

Important Note

- If used in multiple files:
    - static function is not visible outside its file

Important Concepts

- Static variables are initialized once
- Value is preserved between calls
- Stored in data segment (not stack)

Common Mistake

- static is not constant
- It only controls scope and lifetime

4)Extern Storage Class (extern)

Definition

- extern is used to declare a global variable defined in another file or location
- It tells the compiler: *“this variable exists somewhere else”*

Key Properties

1. Lifetime

- Exists for the entire program execution

2. Scope

- Global scope
- Can be accessed across multiple files

3. Default Value

- Initialized to 0 (like global variables)

Important Concepts

- extern does not allocate memory
- It only declares the variable
- Actual memory is allocated where the variable is defined

Example

```c
#include <stdio.h>
int x = 10;   // definition

void fun()
{
   extern int x;   // declaration
   printf("%d\n", x);
}

int main()
{
   fun();
   return 0;
}
```

Output

10

Example (Multiple Files Concept)

File1.c

```c
#include <stdio.h>

int x = 100;   // definition
```

File2.c

```c
#include <stdio.h>

extern int x;   // declaration

int main()
{
   printf("%d", x);
   return 0;
}
```

Explanation

- x is defined in File1
- extern int x; in File2 tells compiler:
    - “Use the same x from another file”
- Both files share the same variable

Important Notes

- Used for sharing global variables between files
- Avoids duplicate memory allocation
- Helps in modular programming

Difference Between auto, register, extern, and static in C

Parameter	auto	register	static	extern
Keyword	auto	register	static	extern

Lifetime	Till function execution / block	Till function execution	Entire program execution	Entire program execution
Scope	Local/ block	Local/block	Local or Global / limited that file	Global / multiple files
Storage/Memory	RAM (Stack)	CPU Register (if available)	Data/BSS Segment	Data/BSS Segment
Default Initial Value	Garbage value	Garbage value	0	0
Memory Section	Stack	CPU Register / Stack	Data or BSS	Data or BSS
Address can be taken?	Yes	No	Yes	Yes
Visibility	Inside block/function only	Inside block/function only	Depends on declaration	Accessible across files
Storage Duration	Automatic	Automatic	Static	Static

Practice questions

1. Default storage class of local variable is:

A) static
B) extern
C) auto
D) register

2. Where are auto variables stored?

A) Heap
B) Stack
C) Data segment
D) Register

3. Default value of auto variable is:

A) 0
B) 1
C) Garbage
D) NULL

4. Which storage class is used for faster access?

A) static
B) extern
C) register
D) auto

5. What is the default value of static variable?

A) Garbage
B) 0
C) 1
D) Undefined

6. Which storage class retains value between function calls?

A) auto
B) register
C) static
D) extern

7. extern keyword is used to:

A) Define variable
B) Declare variable
C) Allocate memory
D) Delete variable

8. Scope of global static variable is:

A) Entire program
B) Block only
C) File only
D) Function only

9. What is the output?

```c
#include <stdio.h>
void fun()
{
   static int x;
   x++;
   printf("%d ", x);
}
int main()
{
   fun();
   fun();
   fun();
}
```

A) 1 1 1
B) 0 1 2
C) 1 2 3
D) Garbage

10. What is the output?

```c
#include <stdio.h>
int fun()
{
   auto int x = 0;
   x++;
   return x;
```

```
}
int main()
{
   printf("%d %d", fun(), fun());
}
```

A) 1 1
B) 1 2
C) 2 2
D) Garbage

11. What will happen?

```
#include <stdio.h>
int main()
{
   register int x = 5;
   printf("%d", &x);
}
```

A) Prints address
B) Compile-time error
C) Runtime error
D) Garbage

12. Identify correct statement:

A) static variable is constant
B) extern allocates memory
C) register guarantees CPU storage
D) static retains value

Objective Questions

1. Define **storage class in C**
2. What is **lifetime of static variable**?
3. What is **scope of extern variable**?
4. Why register variables are used?
5. What is default value of global variable?
6. Can we use & with register variable? Why?
7. Difference between **auto and static**

8. What is meant by **scope of variable**?

Subjective Questions

1. Explain **auto storage class** with example.
2. Explain **register storage class** with example
3. Explain **static storage class** (local and global) with example
4. Explain **extern storage class** with example
5. Compare **auto and static variables**
6. Explain **scope and lifetime** with example
7. Explain all **four storage classes with comparison table**
8. Write a program to demonstrate **static variable behavior**
9. Write a program to show **extern variable usage across files**
10. Explain **memory allocation of variables (stack, data segment)**
11. Write a program showing **difference between auto, static, register**